

Compreensões de Lista

COMP0496 – Programação A

Prof. Dr. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
`andreyoshiaki@dcomp.ufs.br`

Queremos elevar ao quadrado cada número de uma lista.

Com loop explícito:

```
1  numeros = [1, 2, 3, 4, 5]
2  quadrados = []
3  for n in numeros:
4      quadrados.append(n ** 2)
5  print(quadrados)  # [1, 4, 9, 16, 25]
```

Com compreensão de lista:

```
1  quadrados = [n ** 2 for n in numeros]
2  print(quadrados)  # [1, 4, 9, 16, 25]
```

Mais conciso e igualmente legível.

List Comprehension

[expressão for variável in sequência]

Equivale a: criar lista vazia, iterar sobre sequência e fazer `.append(expressão)`.

```
1  # Nomes em maiusculo
2  nomes = ["ana", "bob", "carlos"]
3  maiusculos = [n.upper() for n in nomes]
4  print(maiusculos)  # ['ANA', 'BOB', 'CARLOS']
5
6  # Dobro de cada elemento
7  dobros = [x * 2 for x in range(1, 6)]
8  print(dobros)  # [2, 4, 6, 8, 10]
```

Adicionar `if` ao final seleciona apenas os elementos que satisfazem a condição:
[expressão for variável in sequência if condição]

```
1  # Numeros pares de 1 a 20
2  pares = [x for x in range(1, 21) if x % 2 == 0]
3  print(pares)  # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
4
5  # Notas de aprovados (nota >= 5.0)
6  notas = [3.5, 7.0, 4.8, 9.2, 5.0, 2.1]
7  aprovados = [n for n in notas if n >= 5.0]
8  print(aprovados)  # [7.0, 9.2, 5.0]
```

`{chave: valor for variável in sequência}`

```
1  # Quadrados como dicionario: numero -> quadrado
2  quadrados = {x: x**2 for x in range(1, 6)}
3  print(quadrados)
4  # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
5
6  # Inverter dicionario: valor -> chave
7  original = {"Ana": 1001, "Bob": 1002, "Carlos": 1003}
8  invertido = {v: k for k, v in original.items()}
9  print(invertido)
10 # {1001: 'Ana', 1002: 'Bob', 1003: 'Carlos'}
```

`{expressão for variável in sequência}`

O resultado é um conjunto: sem duplicatas e sem ordem garantida.

```
1  # Letras unicas de uma palavra
2  palavra = "programacao"
3  letras = {c for c in palavra}
4  print(letras)
5  # {'p', 'r', 'o', 'g', 'a', 'm', 'c'} (ordem varia)
6
7  # Primeiras letras dos nomes (unicas)
8  nomes = ["Ana", "Bob", "Alice", "Carlos", "Bruno"]
9  iniciais = {n[0] for n in nomes}
10 print(iniciais) # {'A', 'B', 'C'}
```

Use compreensão quando:

- A transformação é simples e cabe em uma linha
- O objetivo é criar uma nova coleção
- O código fica mais legível assim

Use loop explícito quando:

- Há múltiplas operações por elemento
- Existem efeitos colaterais (ex: escrita em arquivo)
- A lógica é complexa demais para uma linha

Regra prática: se for necessário explicar o que a compreensão faz, use um loop.

- **List comprehension:** `[expr for x in seq]` substitui loop + append.
- Adicionar `if cond` ao final filtra os elementos.
- `{k: v for ...}` cria dicionário; `{expr for ...}` cria conjunto.
- Use compreensão para transformações simples; loop para lógica complexa.